University of Asia Pacific

Department of Computer Science and Engineering

CSE 450: VLSI Lab

LAB REPORT

Experiment Number: 02

Experiment Title: Design and Simulation of an 8-bit Full Adder Using VHDL

Submitted by:

Name: Nazmul Mridha

Student ID: 22201161

Section: C2

Submitted to:

Md. Rasheduzzaman
Lecturer
Department of Computer Science and Engineering

Date of Submission: 03/10/2026

1. Experiment Name

Design and Simulation of an 8-bit Full Adder Using VHDL in Xilinx ISE

2. Objective

The goals of this experiment are:

- To design an 8-bit full adder in VHDL by connecting eight 1-bit full adders (structural modeling).
- To understand how the carry ripples from one stage to the next, where the carry-out of one stage feeds the carry-in of the next.
- To generate the RTL schematic with XST synthesis and verify the design with a test bench in ISim behavioral simulation.

3. Apparatus / Hardware & Software Requirements

- Xilinx ISE Design Suite, P.20131013 (design entry, XST synthesis and ISim simulator)
- Target device: xc7a100t-3csg324
- VHDL with component instantiation (structural modeling)
- Oracle VirtualBox virtual machine

4. Circuit Diagram / Schematic

The module full_adder_eight_bit has 17 inputs (a0–a7, b0–b7 and cin) and 9 outputs (s0–s7 and cout). It is built from eight copies of the 1-bit full adder (one_bit_full_adder) designed in Experiment 1. This arrangement is called a ripple-carry adder. The first stage, FA0, takes the external carry-in cin. The carry-out of each stage becomes the carry-in of the next stage through the internal signals c1 to c7, and the last stage, FA7, produces the final carry-out cout. Each stage produces one sum bit (s0 to s7).

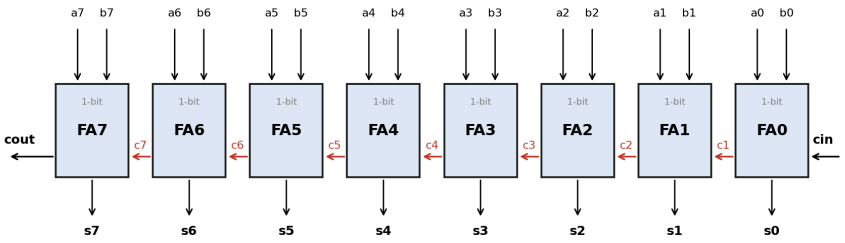


Fig 1: Block diagram of the 8-bit ripple-carry adder

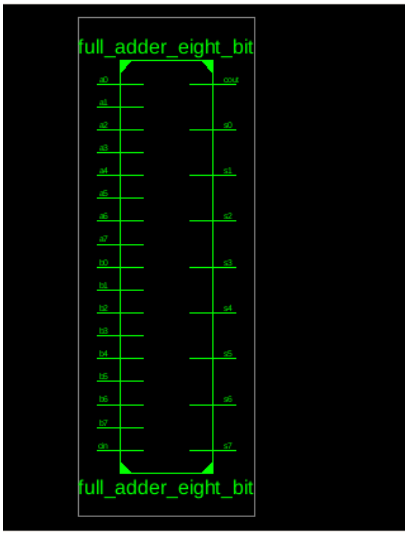


Fig 2: Block symbol of full_adder_eight_bit

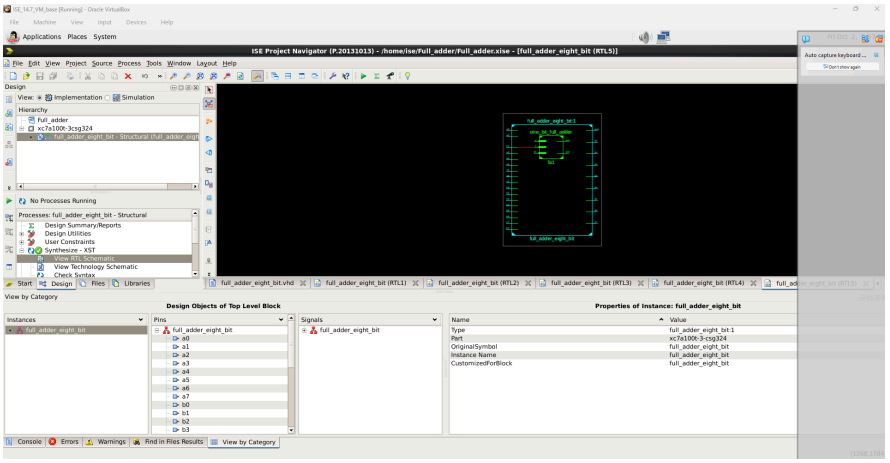


Fig 3: RTL schematic of full_adder_eight_bit in ISE Project Navigator

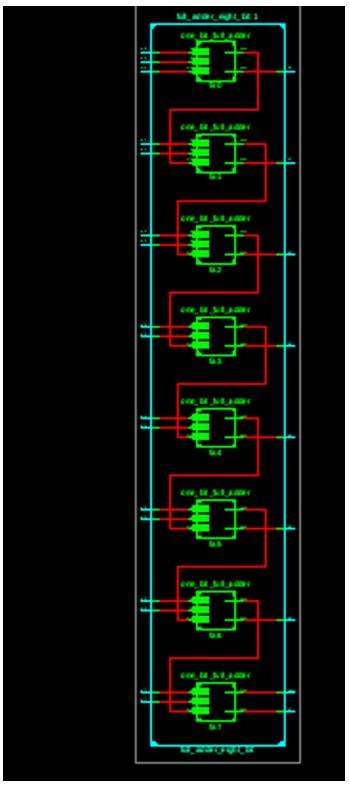


Fig 4: RTL schematic, internal view showing the chain of eight 1-bit full adders

The schematic confirms the structure in Fig 1: eight one_bit_full_adder instances are connected in a chain, with the carry (red wires) passing from each stage to the next.

5. VHDL Source Code

5.1 Design File (full_adder_eight_bit.vhd)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity full_adder_eight_bit is
    Port ( a0 : in  STD_LOGIC;
           a1 : in  STD_LOGIC;
           a2 : in  STD_LOGIC;
           a3 : in  STD_LOGIC;
           a4 : in  STD_LOGIC;
           a5 : in  STD_LOGIC;
           a6 : in  STD_LOGIC;
           a7 : in  STD_LOGIC;
           b0 : in  STD_LOGIC;
           b1 : in  STD_LOGIC;
           b2 : in  STD_LOGIC;
           b3 : in  STD_LOGIC;
           b4 : in  STD_LOGIC;
           b5 : in  STD_LOGIC;
           b6 : in  STD_LOGIC;
           b7 : in  STD_LOGIC;
           cin : in  STD_LOGIC;
           s0 : out  STD_LOGIC;
           s1 : out  STD_LOGIC;
           s2 : out  STD_LOGIC;
           s3 : out  STD_LOGIC;
           s4 : out  STD_LOGIC;
           s5 : out  STD_LOGIC;
           s6 : out  STD_LOGIC;
           s7 : out  STD_LOGIC;
           cout : out  STD_LOGIC);
end full_adder_eight_bit;

architecture Structural of full_adder_eight_bit is

    component one_bit_full_adder
        Port ( a : in  STD_LOGIC;
               b : in  STD_LOGIC;
               cin : in  STD_LOGIC;
               sum : out  STD_LOGIC;
               cout : out  STD_LOGIC);
    end component;

    signal c1,c2,c3,c4,c5,c6,c7 : STD_LOGIC;

begin
    fa0 : one_bit_full_adder port map (a=>a0,b=>b0,cin=>cin,sum=>s0,cout=>c1);
    fa1 : one_bit_full_adder port map (a=>a1,b=>b1,cin=>c1,sum=>s1,cout=>c2);
    fa2 : one_bit_full_adder port map (a=>a2,b=>b2,cin=>c2,sum=>s2,cout=>c3);
    fa3 : one_bit_full_adder port map (a=>a3,b=>b3,cin=>c3,sum=>s3,cout=>c4);
    fa4 : one_bit_full_adder port map (a=>a4,b=>b4,cin=>c4,sum=>s4,cout=>c5);
    fa5 : one_bit_full_adder port map (a=>a5,b=>b5,cin=>c5,sum=>s5,cout=>c6);
    fa6 : one_bit_full_adder port map (a=>a6,b=>b6,cin=>c6,sum=>s6,cout=>c7);
    fa7 : one_bit_full_adder port map (a=>a7,b=>b7,cin=>c7,sum=>s7,cout=>cout);

end Structural;
```

5.2 Test Bench (full_adder_eight_bit_tb.vhd)

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY full_adder_eight_bit_tb IS
END full_adder_eight_bit_tb;

ARCHITECTURE behavior OF full_adder_eight_bit_tb IS

    -- Component Declaration for the Unit Under Test (UUT)
    COMPONENT full_adder_eight_bit
    PORT(
         a0, a1, a2, a3, a4, a5, a6, a7 : IN  std_logic;
         b0, b1, b2, b3, b4, b5, b6, b7 : IN  std_logic;
         cin : IN  std_logic;
         s0, s1, s2, s3, s4, s5, s6, s7 : OUT std_logic;
         cout : OUT std_logic
        );
    END COMPONENT;

    --Inputs
    signal a0, a1, a2, a3, a4, a5, a6, a7 : std_logic := '0';
    signal b0, b1, b2, b3, b4, b5, b6, b7 : std_logic := '0';
    signal cin : std_logic := '0';

    --Outputs
    signal s0, s1, s2, s3, s4, s5, s6, s7 : std_logic;
    signal cout : std_logic;

BEGIN

    -- Instantiate the Unit Under Test (UUT)
    uut: full_adder_eight_bit PORT MAP (
          a0 => a0, a1 => a1, a2 => a2, a3 => a3,
          a4 => a4, a5 => a5, a6 => a6, a7 => a7,
          b0 => b0, b1 => b1, b2 => b2, b3 => b3,
          b4 => b4, b5 => b5, b6 => b6, b7 => b7,
          cin => cin,
          s0 => s0, s1 => s1, s2 => s2, s3 => s3,
          s4 => s4, s5 => s5, s6 => s6, s7 => s7,
          cout => cout
        );

    -- Stimulus process
    stim_proc: process
    begin

        -- 10 + 12
        a0 <= '0'; a1 <= '1'; a2 <= '0'; a3 <= '1';
        a4 <= '0'; a5 <= '0'; a6 <= '0'; a7 <= '0';
        b0 <= '0'; b1 <= '0'; b2 <= '1'; b3 <= '1';
        b4 <= '0'; b5 <= '0'; b6 <= '0'; b7 <= '0';
        cin <= '0';
        wait for 20 ns;

        -- 13 + 14
        a0 <= '1'; a1 <= '0'; a2 <= '1'; a3 <= '1';
        a4 <= '0'; a5 <= '0'; a6 <= '0'; a7 <= '0';
        b0 <= '0'; b1 <= '1'; b2 <= '1'; b3 <= '1';
        b4 <= '0'; b5 <= '0'; b6 <= '0'; b7 <= '0';
        cin <= '0';
        wait for 20 ns;

        -- 9 + 6
        a0 <= '1'; a1 <= '0'; a2 <= '0'; a3 <= '1';
        a4 <= '0'; a5 <= '0'; a6 <= '0'; a7 <= '0';
        b0 <= '0'; b1 <= '1'; b2 <= '1'; b3 <= '0';
        b4 <= '0'; b5 <= '0'; b6 <= '0'; b7 <= '0';
        cin <= '0';
        wait for 20 ns;

        -- 11 + 9
        a0 <= '1'; a1 <= '1'; a2 <= '0'; a3 <= '1';
        a4 <= '0'; a5 <= '0'; a6 <= '0'; a7 <= '0';
        b0 <= '1'; b1 <= '0'; b2 <= '0'; b3 <= '1';
        b4 <= '0'; b5 <= '0'; b6 <= '0'; b7 <= '0';
```

```
        cin <= '0';
        wait for 20 ns;

        -- 14 + 2
        a0 <= '0'; a1 <= '1'; a2 <= '1'; a3 <= '1';
        a4 <= '0'; a5 <= '0'; a6 <= '0'; a7 <= '0';
        b0 <= '0'; b1 <= '1'; b2 <= '0'; b3 <= '0';
        b4 <= '0'; b5 <= '0'; b6 <= '0'; b7 <= '0';
        cin <= '0';
        wait for 20 ns;

        wait;
    end process;

END;
```

6. Output Analysis / Observations

Behavioral simulation of the 8-bit full adder was run in ISim with 20 ns per step. The test bench applies five input vectors with Cin = 0, and the last vector is held until the end of the simulation. The applied inputs and the resulting outputs are listed in Table 1. The waveform in Fig 5 shows the inputs b0–b7 changing every 20 ns while the sum bits s0–s7 follow them.

Table 1: Simulation results of the 8-bit full adder

Time	A (A7–A0)	B (B7–B0)	Cin	Sum (S7–S0)	Cout	A + B + Cin
0 – 20 ns	00001010	00001100	0	00010110	0	10 + 12 + 0 = 22
20 – 40 ns	00001101	00001110	0	00011011	0	13 + 14 + 0 = 27
40 – 60 ns	00001001	00000110	0	00001111	0	9 + 6 + 0 = 15
60 – 80 ns	00001011	00001001	0	00010100	0	11 + 9 + 0 = 20
80 – 100 ns	00001110	00000010	0	00010000	0	14 + 2 + 0 = 16

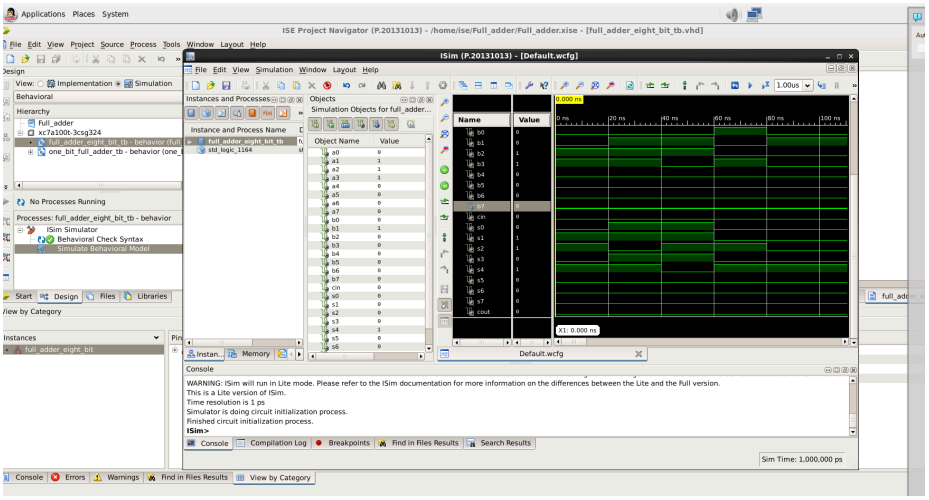


Fig 5: ISim waveform of the 8-bit full adder

Observations

- For every vector the output equals A + B + Cin in binary: 22, 27, 15, 20 and 16.
- At 0–20 ns (10 + 12) and 20–40 ns (13 + 14) carries are generated in the middle bits and move up into the higher sum bits.
- At 40–60 ns (9 + 6) no bit position produces a carry, since 1001 and 0110 have no 1s in the same position, so the sum is 00001111.
- At 80–100 ns (14 + 2) the carry travels through bits 1, 2 and 3 in turn, and the result 00010000 appears only after the carry has rippled through those stages. This is the ripple-carry behavior.
- Cout stays 0 in all five vectors because every result is below 256.

7. Result

The 8-bit full adder was built from eight 1-bit full adders in VHDL (structural model), synthesized with XST and simulated in ISim. For all five test vectors the outputs S7–S0 and Cout matched A + B + Cin. The RTL schematic showed the expected chain of eight full adders with the carry passing from each stage to the next. The objective of the experiment was achieved.

8. GitHub Repository Commits

Repository commits link: https://github.com/Nazmul-SWE/VLSI

9. Conclusion

In this experiment I designed an 8-bit adder by connecting eight 1-bit full adders in VHDL. I learned how to:

- Use component declaration and port mapping to build a larger design from a smaller module (structural modeling).
- Connect the internal signals c1 to c7 so that each stage's carry-out feeds the next stage's carry-in.
- Verify a multi-bit design with a test bench and check the outputs against the expected binary sums.

The experiment also showed the main limitation of a ripple-carry adder: each stage must wait for the carry from the stage below, so the delay grows with the number of bits. It is worst when a carry has to travel through all eight stages. Faster designs such as the carry-lookahead adder reduce this delay.